

# Variationally Consistent Steklov Merging for Modal Poisson Solves on Tetrahedra

(with HDG/Nitsche-type stabilization in moment space)

Sachin Natesh

## 1 Overview and key principle

This note documents the discrete method implemented in two codes: (i) a robust single-tetrahedron Poisson solver using a stacked overdetermined system solved by sparse QR, and (ii) a two-tetrahedron merge constructed by eliminating volume coefficients and solving a face-only interface problem. The central insight is:

*The merge must be the Euler–Lagrange condition of the same discrete functional that defines the stable single-tetrahedron solve.*

A naive Steklov merge (imposing only “flux continuity” via an unstabilized DtN map) fails because it does *not* correspond to stationarity of the underlying stacked least-squares functional; it produces a rank-degenerate (or numerically rank-degenerate) interface operator with a large near-nullspace. A variationally consistent stabilization fixes this by penalizing the mismatch between the *induced* trace (produced by the local least-squares extension) and the *skeleton* trace (the global interface unknown).

## 2 Geometry and bases

Let  $\widehat{K}$ ,  $K$  be the reference and physical tetrahedron, respectively. We have the standard affine map

$$x = \Phi_K(\widehat{x}) = V_0 + J_K \widehat{x}, \quad J_K \in \mathbb{R}^{3 \times 3}.$$

Faces are denoted  $\widehat{F}$  and  $F = \Phi_K(\widehat{F})$ . For a face  $F$ , let  $n_F$  be the outward unit normal.

### 2.1 Volume basis

Fix polynomial degree  $n$ . Let

$$V_n(K) = \mathbb{P}_n(K), \quad M = \dim V_n(K),$$

with modal basis  $\{\phi_j\}_{j=1}^M$  (Jacobi-type on the tetrahedron). We represent the discrete solution as

$$u_h(x) = \sum_{j=1}^M c_j \phi_j(x), \quad c \in \mathbb{R}^M.$$

## 2.2 Face test/trace basis (moment space)

On a face  $F$ , define

$$\Lambda_n(F) = \mathbb{P}_n(F), \quad k_f = \dim \Lambda_n(F),$$

with face test basis  $\{\psi_i\}_{i=1}^{k_f}$  (Jacobi-type on the triangle).

The *face moment (trace) vector* associated with  $u_h$  is

$$\lambda_F(c) \in \mathbb{R}^{k_f}, \quad (\lambda_F(c))_i = \int_F \psi_i u_h ds.$$

This is linear in  $c$ , so there exists a matrix  $T_F \in \mathbb{R}^{k_f \times M}$  such that

$$\lambda_F(c) = T_F c, \quad (T_F)_{ij} = \int_F \psi_i \phi_j ds.$$

Stacking all four faces of a tetrahedron  $K$ ,

$$T_{\text{full}} c = \lambda, \quad T_{\text{full}} \in \mathbb{R}^{(4k_f) \times M}, \quad \lambda = (\lambda_{F_0}, \dots, \lambda_{F_3}).$$

## 3 Promoted interior PDE constraints and their weights

Let  $W_{n-2}(K) = \mathbb{P}_{n-2}(K)$  and choose an interior test basis  $\{q_i\}_{i=1}^m$ ,  $m = \dim W_{n-2}(K)$ . The method uses *promoted* constraints of the form

$$\int_K (\Delta u_h + f) q_i w_1(x) dx = 0, \quad i = 1, \dots, m. \quad (1)$$

**Discrete interior operator.** Define  $L_{\text{int}} \in \mathbb{R}^{m \times M}$  and  $f_{\text{int}} \in \mathbb{R}^m$  by

$$(L_{\text{int}})_{ij} = \int_K (\Delta \phi_j) q_i w_1(x) dx, \quad (f_{\text{int}})_i = \int_K f q_i w_1(x) dx, \quad (2)$$

where  $w_1(x)$  is the weight function under which the promoted basis is orthonormal. Then (1) becomes

$$L_{\text{int}} c = -f_{\text{int}}. \quad (3)$$

**Remark 1** (Weights and quadrature). *In practice, the integrals in (2) are computed by quadrature on  $\hat{K}$  and mapped to  $K$ . The promoted operator is assembled in the same moment/test basis as used to project the source  $f$ . That is,  $f_{\text{int}}$  is not arbitrary; it is the vector of moments of  $f$  against  $\{q_i\}$  with the physical volume measure  $dx$ . This basis/weight consistency is crucial for polynomial exactness in the manufactured-solution tests.*

## 4 The single-tetrahedron solve as a variational problem

The stable single-tet method is defined by minimizing the stacked residual functional

$$\mathcal{J}_K(c; \lambda, f) = \frac{1}{2} \|T_{\text{full}} c - \lambda\|_2^2 + \frac{1}{2} \|L_{\text{int}} c + f_{\text{int}}\|_2^2. \quad (4)$$

Equivalently,

$$\mathcal{J}_K(c; \lambda, f) = \frac{1}{2} \|Ac - b(\lambda, f)\|_2^2, \quad A = \begin{bmatrix} T_{\text{full}} \\ L_{\text{int}} \end{bmatrix}, \quad b = \begin{bmatrix} \lambda \\ -f_{\text{int}} \end{bmatrix}. \quad (5)$$

The computed coefficients are

$$c^*(\lambda, f) = \arg \min_{c \in \mathbb{R}^M} \mathcal{J}_K(c; \lambda, f). \quad (6)$$

#### 4.1 QR construction of the solution map

The code computes  $c^*$  using sparse QR (SuiteSparseQR) of the (sparse) stacked matrix  $A$ . Let

$$AE = QR,$$

where  $E$  is a column permutation,  $Q$  has orthonormal columns, and  $R$  is upper triangular on the numerical rank. Then the minimum-norm least-squares solution is obtained by

$$c^* = E \begin{bmatrix} R_{11}^{-1}(Q^\top b)_{1:r} \\ 0 \end{bmatrix}, \quad (7)$$

with numerical rank  $r$  (matching the code's SPQR bookkeeping).

**Linearity.** Because  $c^*$  is computed by a fixed linear pseudoinverse  $A^+$ ,

$$c^*(\lambda, f) = A^+ \begin{bmatrix} \lambda \\ -f_{\text{int}} \end{bmatrix},$$

hence  $c^*$  is affine in the imposed boundary moments  $\lambda$  and in the interior forcing moments  $f_{\text{int}}$ .

### 5 Flux moments and trace moments on a chosen face

Fix a face  $\Gamma \subset \partial K$ . Define

$$T_\Gamma \in \mathbb{R}^{k_f \times M} \quad (\text{trace moment map on } \Gamma), \quad F_\Gamma \in \mathbb{R}^{k_f \times M} \quad (\text{flux moment map on } \Gamma).$$

The *induced trace* (from volume coefficients) is

$$\lambda_\Gamma^{\text{ind}}(c) = T_\Gamma c,$$

and the *flux moments* are

$$\mu_\Gamma(c) = F_\Gamma c, \quad (F_\Gamma)_{ij} = \int_\Gamma \psi_i (\nabla \phi_j \cdot n_\Gamma) ds.$$

(Importantly,  $F_\Gamma$  must be built using the physical gradient transform  $J_K^{-T}$  and the physical unit normal; this is why the final corrected implementation builds  $F_\Gamma$  in each leaf, not by scaling a reference precompute.)

### 6 Local affine response operators (precise construction)

Consider one tetrahedron  $K$  and one face  $\Gamma$ . We split the boundary moments into:

$$\lambda = (\lambda_\Gamma, \lambda_{\text{ext}}),$$

where  $\lambda_{\text{ext}}$  collects the remaining three face blocks.

Fix  $(\lambda_{\text{ext}}, f_{\text{int}})$  and view the solution as a function of  $\lambda_\Gamma$  only.

## 6.1 Block injection and solve-many

Let  $B_\Gamma \in \mathbb{R}^{(4k_f+m) \times k_f}$  be the injection matrix that writes a  $k_f$ -vector into the  $\Gamma$  rows of the stacked right-hand side (and zeros elsewhere). In code this is `B_face(iface)`.

Define the particular solve (interface set to zero):

$$c_0 = c^*(\lambda_\Gamma = 0, \lambda_{\text{ext}}, f_{\text{int}}). \quad (8)$$

Define the homogeneous response matrix

$$C_\Gamma = c^*(\lambda_\Gamma = I, \lambda_{\text{ext}} = 0, f_{\text{int}} = 0) = A^+ B_\Gamma \in \mathbb{R}^{M \times k_f}, \quad (9)$$

computed by applying the same QR solve to the multi-right-hand-side matrix  $B_\Gamma$ .

Then the general solution is

$$c^*(\lambda_\Gamma, \lambda_{\text{ext}}, f_{\text{int}}) = C_\Gamma \lambda_\Gamma + c_0. \quad (10)$$

## 6.2 Derived face operators

From (10),

$$\mu_\Gamma(\lambda_\Gamma) = F_\Gamma c^* = (F_\Gamma C_\Gamma) \lambda_\Gamma + F_\Gamma c_0 =: S_F \lambda_\Gamma + g_F, \quad (11)$$

$$\lambda_\Gamma^{\text{ind}}(\lambda_\Gamma) = T_\Gamma c^* = (T_\Gamma C_\Gamma) \lambda_\Gamma + T_\Gamma c_0 =: S_T \lambda_\Gamma + g_T. \quad (12)$$

These are the *local affine response operators*:

$$S_F \in \mathbb{R}^{k_f \times k_f}, \quad g_F \in \mathbb{R}^{k_f}, \quad S_T \in \mathbb{R}^{k_f \times k_f}, \quad g_T \in \mathbb{R}^{k_f}.$$

**Remark 2** (Interpretation of  $S_T$ ). *In a classical Dirichlet solve one would have  $\lambda_\Gamma^{\text{ind}}(\lambda_\Gamma) = \lambda_\Gamma$ , i.e.  $S_T = I$  and  $g_T = 0$ . In the present method,  $c^*$  is a least-squares extension balancing interior and trace residuals, so in general  $S_T \neq I$ . This mismatch is precisely what must be accounted for in a variationally consistent merge.*

## 7 Two-tetrahedron setting and skeleton vs induced trace

Let  $A$  and  $B$  be two tetrahedra sharing an interface face  $\Gamma$ . Let  $\lambda \in \mathbb{R}^{k_f}$  denote the *skeleton trace moments* on  $\Gamma$ , i.e. the global interface unknown.

For each element, solving the local problem with skeleton input  $\lambda$  produces a volume coefficient vector  $c_A(\lambda)$  and  $c_B(\lambda)$ , hence an *induced trace* on  $\Gamma$ :

$$\lambda_A^{\text{ind}}(\lambda) = T_\Gamma^A c_A(\lambda), \quad \lambda_B^{\text{ind}}(\lambda) = T_\Gamma^B c_B(\lambda).$$

In a perfectly conforming method these would equal  $\lambda$  identically; in the stacked LS method they do not unless  $\lambda$  lies in a compatible subspace.

## 8 Why the naive Steklov merge is ill-conditioned / rank-degenerate

The naive merge enforces physical flux continuity using the (unstabilized) DtN maps:

$$\mu_A(\lambda) + \mu_B(\lambda) = 0 \iff (S_F^A + S_F^B) \lambda = -(g_F^A + g_F^B). \quad (13)$$

## 8.1 Mechanism: near-nullspace from smoothing

The local least-squares extension  $c^*(\lambda)$  is a discrete harmonic extension operator in high-order face modes: high-degree face moments can be extended into the volume with very small interior energy, producing very small normal flux. Thus  $S_F$  has many small singular values and may be numerically rank-deficient:

$$S_F \approx 0 \quad \text{on a large subspace of face modes.}$$

Consequently,

$$K_{\text{naive}} := S_F^A + S_F^B$$

inherits a large near-nullspace and becomes severely ill-conditioned (or numerically rank-deficient). This is exactly what the observed spectra show: a smooth decay followed by a cluster at machine precision.

## 8.2 Mismatch with the discrete variational structure

More fundamentally, (13) is *not* the stationarity condition of any reduced functional obtained by eliminating  $c_A, c_B$  from the true discrete objective (4). As a result, it does not select the interface trace consistent with the local least-squares extension, leading to loss of accuracy (observed plateau).

# 9 Variationally consistent stabilization (HDG/Nitsche in moment space)

## 9.1 Augmented numerical flux as a penalty on induced–skeleton mismatch

We define the augmented (numerical) flux moments on  $\Gamma$  for each element:

$$\hat{\mu}(\lambda) = \mu(\lambda) + \tau M_\Gamma (\lambda^{\text{ind}}(\lambda) - \lambda), \quad (14)$$

where  $M_\Gamma$  is the face mass matrix in the same moment basis:

$$(M_\Gamma)_{ij} = \int_\Gamma \psi_i \psi_j \, ds.$$

Here  $\tau > 0$  penalizes the mismatch between the induced trace and the skeleton trace in the  $L^2(\Gamma)$  moment metric.

Using (11)–(12),

$$\hat{\mu}(\lambda) = (S_F + \tau M_\Gamma (S_T - I))\lambda + (g_F + \tau M_\Gamma g_T). \quad (15)$$

## 9.2 Stabilized merge equation

Flux continuity is imposed on the augmented flux:

$$\hat{\mu}_A(\lambda) + \hat{\mu}_B(\lambda) = 0. \quad (16)$$

This yields the stabilized interface system

$$K_{\text{stab}}\lambda = -h_{\text{stab}}, \quad (17)$$

with

$$K_{\text{stab}} = (S_F^A + \tau M(S_T^A - I)) + (S_F^B + \tau M(S_T^B - I)), \quad (18)$$

$$h_{\text{stab}} = (g_F^A + \tau M g_T^A) + (g_F^B + \tau M g_T^B). \quad (19)$$

(When element face bases differ, moment vectors are mapped by an orthonormal face change-of-basis  $P$ ; the above is written in a common coordinate convention.)

## 10 Discrete functional for two tetrahedra and Euler–Lagrange derivation

### 10.1 Global discrete objective (skeleton-coupled)

Define the global functional

$$\begin{aligned} \mathcal{J}(c_A, c_B, \lambda) = & \frac{1}{2} \|T_A c_A - \lambda_A^{\text{data}}\|^2 + \frac{1}{2} \|L_A c_A + f_A\|^2 + \frac{1}{2} \|T_B c_B - \lambda_B^{\text{data}}\|^2 + \frac{1}{2} \|L_B c_B + f_B\|^2 \\ & + \frac{\tau}{2} \|\Pi_\Gamma T_\Gamma^A c_A - \lambda\|_M^2 + \frac{\tau}{2} \|\Pi_\Gamma T_\Gamma^B c_B - \lambda\|_M^2. \end{aligned} \quad (20)$$

Here:

- $T_A, T_B$  are the full face-trace operators for each tet (all faces).
- $\lambda_A^{\text{data}}, \lambda_B^{\text{data}}$  encode the prescribed exterior Dirichlet moments (with zeros on the interface block).
- $\Pi_\Gamma$  extracts the interface face block from the stacked face vector (equivalently, we can use  $T_\Gamma$  directly).
- $\|v\|_M^2 := v^\top M v$  is the face-mass norm on  $\Gamma$ .

**Meaning.** The new terms enforce that the induced trace on the interface is close to the skeleton trace  $\lambda$ . They are the *only* modification required to make hybridization variationally consistent with the local least-squares solves.

### 10.2 Elimination of volume variables and reduced functional

For a fixed  $\lambda$ , define

$$(c_A(\lambda), c_B(\lambda)) = \arg \min_{c_A, c_B} \mathcal{J}(c_A, c_B, \lambda).$$

The reduced functional is

$$\mathcal{J}_{\text{red}}(\lambda) = \mathcal{J}(c_A(\lambda), c_B(\lambda), \lambda).$$

### 10.3 Euler–Lagrange equation for $\lambda$

The Euler–Lagrange equation for  $\lambda$  is the stationarity condition

$$\nabla_\lambda \mathcal{J}_{\text{red}}(\lambda) = 0.$$

Since  $\lambda$  appears only in the two penalty terms in (20), and those terms are quadratic, we can compute the derivative directly (envelope theorem: derivatives through  $c_A(\lambda), c_B(\lambda)$  vanish at the minimizer for variations in  $\lambda$ ).

Let

$$r_A(\lambda) = \Pi_\Gamma T_\Gamma^A c_A(\lambda) - \lambda, \quad r_B(\lambda) = \Pi_\Gamma T_\Gamma^B c_B(\lambda) - \lambda.$$

Then

$$\mathcal{J}_{\text{red}}(\lambda) = (\text{terms independent of } \lambda) + \frac{\tau}{2} r_A(\lambda)^\top M r_A(\lambda) + \frac{\tau}{2} r_B(\lambda)^\top M r_B(\lambda).$$

Differentiating with respect to  $\lambda$  gives

$$\nabla_\lambda \mathcal{J}_{\text{red}}(\lambda) = -\tau M r_A(\lambda) - \tau M r_B(\lambda).$$

Setting this to zero yields

$$M(\Pi_\Gamma T_\Gamma^A c_A(\lambda) - \lambda) + M(\Pi_\Gamma T_\Gamma^B c_B(\lambda) - \lambda) = 0. \quad (21)$$

**From trace balance to augmented flux balance.** The stabilized method implemented in code uses *augmented flux continuity* (16). To see equivalence, note that each local minimizer  $c_K(\lambda)$  satisfies the optimality condition for  $c$ :

$$(T_{\text{full}}^K)^\top (T_{\text{full}}^K c_K - \lambda_K^{\text{data}}) + (L_{\text{int}}^K)^\top (L_{\text{int}}^K c_K + f_K) + \tau (T_\Gamma^K)^\top M (T_\Gamma^K c_K - \lambda) = 0.$$

This is the discrete analogue of a Nitsche/HDG formulation, and the term

$$\tau (T_\Gamma^K)^\top M (T_\Gamma^K c_K - \lambda)$$

is precisely the penalty contribution that, when interpreted on the face, produces the additional term  $\tau M(\lambda^{\text{ind}} - \lambda)$  in the numerical flux (14). In other words, the interface condition (21) is equivalent to continuity of the augmented flux, because the discrete weak form on each element couples normal flux and trace mismatch in the same moment metric.

Operationally, after elimination, the resulting reduced linear system for  $\lambda$  is exactly (17) because

$$\mu(\lambda) = S_F \lambda + g_F, \quad \lambda^{\text{ind}}(\lambda) = S_T \lambda + g_T,$$

and substituting into (14) yields (15), hence (17).

**Theorem 1** (Variational-discrete consistency). *The stabilized interface equation (17) is the stationarity condition of the reduced functional  $\mathcal{J}_{\text{red}}(\lambda)$  induced by the global discrete functional (20). Equivalently, the merge respects the same discrete variational structure as the local least-squares solves.*

## 11 Effect of stabilization: coercivity and conditioning

### 11.1 Coercivity intuition

The penalty term adds control of trace mismatch:

$$\frac{\tau}{2} \|\lambda^{\text{ind}}(\lambda) - \lambda\|_M^2.$$

This removes directions in trace space for which the unstabilized DtN map has nearly zero flux response. Those directions were responsible for the near-null singular spectrum of the naive  $K$ .

## 11.2 Spectral effect (why $K$ becomes well-conditioned)

Write the stabilized operator (one element) as

$$K_K(\tau) = S_F^K + \tau M(S_T^K - I).$$

In the ideal conforming case,  $S_T^K = I$  and the penalty vanishes, recovering the classical DtN merge. In the stacked LS method,  $S_T^K - I$  is nontrivial and supplies exactly the missing coercivity on the problematic trace modes.

Empirically (and consistent with the observed SVD plots), the naive operator has many small singular values because  $S_F^K$  is compact/smoothing on high modes. The penalty term contributes an  $O(\tau)$  component in the  $M$ -metric on those modes, lifting the spectrum and eliminating numerical rank deficiency.

## 12 Choice of $\tau$

A robust scaling is

$$\tau = C \frac{(n+1)^2}{h_\Gamma},$$

where  $h_\Gamma$  is a characteristic face diameter (e.g. maximum edge length) and  $C$  is a moderate constant.

## 13 Conclusion

The method is a hybrid Steklov/HDG scheme in moment space: local solves are defined by a stacked least-squares functional, and the interface unknown is obtained by enforcing continuity of an augmented numerical flux that penalizes induced-skeleton trace mismatch. The decisive principle is variational-discrete consistency: the merge must be the Euler–Lagrange equation of the same discrete functional that defines the stable single-tet solve.

### Block sparsity assumptions on Jacobi simplex operators

Let

$$\Pi_n^D = \bigoplus_{j=0}^n \mathcal{H}_j^D$$

denote the decomposition of total-degree polynomials into homogeneous subspaces  $\mathcal{H}_j^D$  of degree exactly  $j$ . We fix, for each  $j$ , a Jacobi simplex basis  $\{\phi_\alpha^{(\kappa)} : |\alpha| = j\}$  of  $\mathcal{H}_j^D$ , ordered by a graded rule. We assume the existence of a canonical bijection between multi-indices  $\alpha$  with  $|\alpha| = j$  and local indices in  $\mathcal{H}_j^D$ .

All operators below are represented in this basis.

**(i) Degree-lowering operators (DMat).** Let

$$\mathcal{D} : \Pi_n^D \rightarrow \Pi_{n-1}^D$$

be a linear operator such that

$$\mathcal{D} \mathcal{H}_j^D \subset \mathcal{H}_{j-1}^D, \quad j \geq 1.$$



Then the matrix of  $\mathcal{D}$  has block structure

$$\mathcal{D} = \begin{pmatrix} 0 & 0 & 0 & \cdots & 0 \\ D_1 & 0 & 0 & \cdots & 0 \\ 0 & D_2 & 0 & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & D_n & 0 \end{pmatrix},$$

where  $D_j : \mathcal{H}_j^D \rightarrow \mathcal{H}_{j-1}^D$ .

*Sparsity assumption.* There exists a finite set

$$\Delta_{\mathcal{D}} \subset \mathbb{Z}^D, \quad \sum_{d=1}^D \Delta_d = -1,$$

independent of  $j$ , such that

$$\mathcal{D}\phi_{\alpha}^{(\kappa)} \in \text{span}\{\phi_{\alpha+\Delta}^{(\kappa')} : \Delta \in \Delta_{\mathcal{D}}\}.$$

**(ii) Degree-filtering operators (KMat).** Let

$$\mathcal{K} : \Pi_n^D \rightarrow \Pi_n^D$$

be a linear operator arising from a *natural promotion* of Jacobi parameters,

$$\kappa_{\text{tgt}} = \kappa_{\text{src}} + e_r, \quad r \in \{0, \dots, D\},$$

corresponding to multiplication of the orthogonality weight by a single barycentric factor. Although  $\mathcal{K}$  preserves the total polynomial degree globally, it is *not* degree-preserving at the level of homogeneous subspaces.

More precisely,  $\mathcal{K}$  satisfies the block filtering property

$$\mathcal{K} \mathcal{H}_j^D \subset \mathcal{H}_j^D \oplus \mathcal{H}_{j-1}^D, \quad j \geq 0,$$

with the convention  $\mathcal{H}_{-1}^D = \{0\}$ . Equivalently, the matrix of  $\mathcal{K}$  in the graded basis

$$\Pi_n^D = \bigoplus_{j=0}^n \mathcal{H}_j^D$$

has a two-band block structure:

$$\mathcal{K} = \begin{pmatrix} A_0 & B_0 & 0 & 0 & \cdots \\ 0 & A_1 & B_1 & 0 & \cdots \\ 0 & 0 & A_2 & B_2 & \cdots \\ \vdots & & \ddots & \ddots & \ddots \end{pmatrix},$$

where

$$A_j : \mathcal{H}_j^D \rightarrow \mathcal{H}_j^D, \quad B_j : \mathcal{H}_{j+1}^D \rightarrow \mathcal{H}_j^D.$$

That is, writing  $u$  as a graded decomposition

$$u = \sum_{j=0}^n u_j, \quad u_j \in \mathcal{H}_j^{\kappa_{\text{src}}},$$

we define the operator

$$\mathcal{K} : \Pi_n^{\kappa_{\text{src}}} \rightarrow \Pi_n^{\kappa_{\text{tgt}}},$$

so that

$$\mathcal{K}u = \sum_{i=0}^n (\mathcal{K}u)_i, \quad (\mathcal{K}u)_i \in \mathcal{H}_i^{\text{tgt}}.$$

So, target block  $i$  receives contributions from source blocks  $i$  and  $i+1$ . Now, if  $u$  only has one homogeneous component,  $u \equiv u_j \in \mathcal{H}_j^{\text{src}}$ , then the only terms remaining in the sum over  $i$  from the formula

$$(\mathcal{K}u)_i = A_i u_i + B_i u_{i+1}$$

are  $i = j$  via  $A_j u_j$  and  $i = j-1$  via  $B_{j-1} u_j$ . Then, we see that

$$\mathcal{K}u_j = (\mathcal{K}u)_j + (\mathcal{K}u)_{j-1} \Rightarrow \mathcal{K}\mathcal{H}_j \subset \mathcal{H}_j \oplus \mathcal{H}_{j-1}.$$

*Sparsity assumption.* There exist finite sets of multi-index offsets

$$\Delta_{\mathcal{K}}^{(0)}, \Delta_{\mathcal{K}}^{(-1)} \subset \mathbb{Z}^D,$$

independent of  $j$ , such that for every basis function  $\phi_{\alpha}^{(\kappa_{\text{src}})}$  with  $|\alpha| = j$ ,

$$\mathcal{K}\phi_{\alpha}^{(\kappa_{\text{src}})} \in \text{span}\left(\{\phi_{\alpha+\Delta}^{(\kappa_{\text{tgt}})} : \Delta \in \Delta_{\mathcal{K}}^{(0)}\} \cup \{\phi_{\alpha+\Delta}^{(\kappa_{\text{tgt}})} : \Delta \in \Delta_{\mathcal{K}}^{(-1)}, |\alpha+\Delta| = j-1\}\right).$$

The sets  $\Delta_{\mathcal{K}}^{(0)}$  and  $\Delta_{\mathcal{K}}^{(-1)}$  encode, respectively, the intra-degree couplings ( $A_j$  blocks) and the one-degree-down couplings ( $B_j$  blocks).

*Stencil discovery.* To determine these coupling patterns, we assemble a dense reference operator  $\mathcal{K}^{(n_{\text{test}})}$  at a small test degree  $n_{\text{test}}$  and restrict it to the representative row block

$$\mathcal{K}_{\text{rep}}^{(n_{\text{test}})} = \mathcal{K}^{(n_{\text{test}})}|_{\mathcal{H}_{j_{\text{rep}}}^D \times (\mathcal{H}_{j_{\text{rep}}}^D \oplus \mathcal{H}_{j_{\text{rep}}+1}^D)}, \quad j_{\text{rep}} = n_{\text{test}} - 1.$$

For each nonzero entry in this restricted block, we record the multi-index difference

$$\Delta = \alpha_{\text{row}} - \alpha_{\text{col}},$$

partitioning the resulting offsets according to whether they correspond to  $\mathcal{H}_j \leftarrow \mathcal{H}_j$  or  $\mathcal{H}_j \leftarrow \mathcal{H}_{j+1}$  couplings. Once these offset sets stabilize as  $n_{\text{test}}$  increases, they are reused to assemble  $\mathcal{K}$  at arbitrary polynomial degree  $n$  without further dense computation.

**(iii) Three-term recurrence operators (JMat).** Let

$$\mathcal{J} : \Pi_n^D \rightarrow \Pi_{n+1}^D$$

be a linear operator satisfying

$$\mathcal{J}\mathcal{H}_j^D \subset \mathcal{H}_{j-1}^D \oplus \mathcal{H}_j^D \oplus \mathcal{H}_{j+1}^D.$$

Then the matrix of  $\mathcal{J}$  has block tridiagonal structure:

$$\mathcal{J} = \begin{pmatrix} B_0 & C_0 & 0 & \cdots & 0 \\ A_1 & B_1 & C_1 & \ddots & \vdots \\ 0 & A_2 & B_2 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & C_{n-1} \\ 0 & \cdots & 0 & A_n & B_n \end{pmatrix},$$

with

$$A_j : \mathcal{H}_j^D \rightarrow \mathcal{H}_{j-1}^D, \quad B_j : \mathcal{H}_j^D \rightarrow \mathcal{H}_j^D, \quad C_j : \mathcal{H}_j^D \rightarrow \mathcal{H}_{j+1}^D.$$

*Sparsity assumption.* There exists a finite set

$$\Delta_{\mathcal{J}} \subset \mathbb{Z}^D, \quad \sum_{d=1}^D \Delta_d \in \{-1, 0, +1\},$$

independent of  $j$ , such that

$$\mathcal{J}\phi_{\alpha}^{(\kappa)} \in \text{span}\{\phi_{\alpha+\Delta}^{(\kappa')} : \Delta \in \Delta_{\mathcal{J}}\}.$$

### Delta-based representation of operator couplings

Let  $\mathcal{O}$  be a linear operator acting on the graded polynomial space

$$\Pi_n^D = \bigoplus_{j=0}^n \mathcal{H}_j^D,$$

where  $\mathcal{H}_j^D$  denotes the subspace of homogeneous polynomials of total degree  $j$  in  $D$  variables. Fix a Jacobi simplex basis  $\{\phi_{\alpha}^{(\kappa)} : |\alpha| = j\}$  for each  $\mathcal{H}_j^D$ , indexed by multi-indices  $\alpha \in \mathbb{N}^D$  with  $|\alpha| = j$ .

**Coupling types.** We assume that the action of  $\mathcal{O}$  on a basis function  $\phi_{\alpha}^{(\kappa)}$  can be written in the form

$$\mathcal{O}\phi_{\alpha}^{(\kappa)} = \sum_{\Delta \in \Delta_{\mathcal{O}}} c_{\Delta}(\alpha) \phi_{\alpha+\Delta}^{(\kappa')},$$

where:

- $\Delta_{\mathcal{O}} \subset \mathbb{Z}^D$  is a finite set of *multi-index shifts* (“deltas”),
- the coefficients  $c_{\Delta}(\alpha)$  may depend on  $\alpha$  and the Jacobi parameters, but
- the set  $\Delta_{\mathcal{O}}$  itself is independent of the polynomial degree  $j$ .

Each  $\Delta \in \Delta_{\mathcal{O}}$  represents a distinct *coupling type* between a source basis function indexed by  $\alpha$  and a target basis function indexed by  $\alpha + \Delta$ .

**Admissibility.** For a fixed source multi-index  $\alpha$ , a shift  $\Delta$  is said to be *admissible* if the target multi-index

$$\beta = \alpha + \Delta$$

satisfies:

1. componentwise nonnegativity,  $\beta_i \geq 0$  for all  $i$ , and
2. the appropriate degree constraint

$$|\beta| = |\alpha| + s,$$

where  $s$  is the degree shift associated with  $\mathcal{O}$ .

Only admissible shifts contribute to the action of the operator at a given  $\alpha$ .

**Stability of the delta set.** For small degrees  $j$ , some coupling types may be absent due to boundary effects in the index set  $\{\alpha : |\alpha| = j\}$ . However, there exists a degree  $j_*$  such that for all  $j \geq j_*$  the set of observed admissible shifts coincides with a fixed finite set  $\Delta_{\mathcal{O}}$ . We refer to this property as *stability* of the delta set.

**Assembly at arbitrary degree.** Once a stable delta set  $\Delta_{\mathcal{O}}$  has been identified, the matrix representation of  $\mathcal{O}$  on  $\Pi_n^D$  for arbitrary  $n$  may be constructed as follows:

1. loop over all basis functions (columns) indexed by  $\alpha$ ,
2. for each  $\alpha$ , loop over all  $\Delta \in \Delta_{\mathcal{O}}$ ,
3. form the candidate target index  $\beta = \alpha + \Delta$ ,
4. if  $\beta$  is admissible, insert the corresponding matrix entry coupling  $\phi_{\alpha}^{(\kappa)}$  to  $\phi_{\beta}^{(\k')}$ .

The mapping between multi-indices and linear matrix indices is provided by the fixed ordering of the Jacobi basis within each homogeneous subspace and depends on the total degree, but the set of admissible shifts  $\Delta_{\mathcal{O}}$  is independent of  $n$ .

**Interpretation.** In this formulation, the delta set  $\Delta_{\mathcal{O}}$  encodes all possible local coupling types induced by the operator in multi-index space. The degree dependence of the matrix sparsity pattern arises solely from the admissibility conditions and the enumeration of basis functions, not from changes in the underlying operator structure. The apparent spreading of nonzeros with increasing degree is a consequence of the growing number of basis functions between two fixed multi-index shifts in the graded ordering; the underlying multi-index coupling encoded by each delta remains unchanged.

### Data-driven discovery of the finite delta sets $\Delta_*$

For each operator family  $\mathcal{O}$  (e.g.  $\mathcal{D}$ ,  $\mathcal{K}$ ,  $\mathcal{J}$ ) we assume the existence of a finite multi-index shift set

$$\Delta_{\mathcal{O}} \subset \mathbb{Z}^D$$

such that, for every homogeneous degree  $j$  and every basis index  $\alpha$  with  $|\alpha| = j$ , the action of  $\mathcal{O}$  on  $\phi_{\alpha}^{(\kappa)}$  is supported only on shifts  $\alpha + \Delta$  with  $\Delta \in \Delta_{\mathcal{O}}$  (cf. the sparsity assumptions above). We describe a practical strategy to discover  $\Delta_{\mathcal{O}}$ .

**Step 0: choose a representative homogeneous block.** Fix a test degree  $n_{\text{test}}$  and select a representative block degree

$$j_{\text{rep}} = j_{\text{rep}}(n_{\text{test}})$$

for extracting the stencil. In our implementation we take

$$j_{\text{rep}} = n_{\text{test}} - 1,$$

i.e. the second highest-degree homogeneous block in  $\Pi_{n_{\text{test}}}^D$ , which is the largest block and empirically exhibits the full interior coupling.

**Step 1: assemble a pruned dense test operator.** Assemble a dense matrix representation

$$O^{(n_{\text{test}})} \in \mathbb{R}^{M \times M}, \quad M = \dim \Pi_{n_{\text{test}}}^D,$$

for the operator  $\mathcal{O}$  (with the desired parameters  $\kappa$ , shifts, and axis), and apply numerical pruning to remove entries below a relative tolerance (e.g. row-relative threshold proportional to machine epsilon). This produces a matrix whose remaining nonzeros reflect the algebraic sparsity pattern.

**Step 2: extract the set of observed deltas from the representative block.** Restrict  $O^{(n_{\text{test}})}$  to the representative homogeneous block

$$O_{\text{rep}}^{(n_{\text{test}})} = O^{(n_{\text{test}})} \big|_{\mathcal{H}_{j_{\text{rep}}}^D \times \mathcal{H}_{j_{\text{rep}}+s}^D},$$

where the degree shift  $s$  is determined by the operator:

$$s = \begin{cases} -1, & \mathcal{O} = \mathcal{D} \\ \in \{-1, 0\}, & \mathcal{O} = \mathcal{K} \\ \in \{-1, 0, +1\}, & \mathcal{O} = \mathcal{J} \text{ (handled per block band)} \end{cases}$$

and where  $\mathcal{H}_{j_{\text{rep}}+s}^D$  denotes the appropriate source block. For each nonzero entry

$$(O_{\text{rep}}^{(n_{\text{test}})})_{i,j} \neq 0,$$

let  $\alpha_i$  and  $\alpha_j$  be the (unique) multi-indices associated with the row and column basis functions in the chosen ordering (existence assumed). Define the observed delta

$$\Delta = \alpha_i - \alpha_j \in \mathbb{Z}^D.$$

Collect all such deltas and retain the unique set:

$$\Delta_{\mathcal{O}}^{(n_{\text{test}})} = \left\{ \alpha_i - \alpha_j : (O_{\text{rep}}^{(n_{\text{test}})})_{i,j} \neq 0 \right\}.$$

**Step 3: stabilization over increasing  $n_{\text{test}}$ .** Starting from a minimal degree  $n_{\text{min}}$  (e.g.  $n_{\text{min}} = 5$ ), repeat Steps 1–2 for  $n_{\text{test}} = n_{\text{min}}, n_{\text{min}} + 1, \dots, n_{\text{max}}$ , producing a sequence of candidate delta sets  $\Delta_{\mathcal{O}}^{(n_{\text{test}})}$ . We declare the stencil to have stabilized when

$$\Delta_{\mathcal{O}}^{(n_{\text{test}})} = \Delta_{\mathcal{O}}^{(n_{\text{test}}-1)}$$

(as sets, e.g. after sorting a canonical encoding), and set

$$\Delta_{\mathcal{O}} := \Delta_{\mathcal{O}}^{(n_{\text{test}})}.$$

**Usage at arbitrary degree.** Once  $\Delta_{\mathcal{O}}$  is known, the operator at any target degree  $n$  can be assembled by looping over source basis functions (multi-indices  $\alpha$ ) and applying the admissible shifts:

$$\alpha \mapsto \alpha + \Delta, \quad \Delta \in \Delta_{\mathcal{O}},$$

discarding invalid targets (e.g. negative components or incorrect total degree), and placing the corresponding nonzero entry in the sparse matrix representation. The conversion between multi-indices and local indices within  $\mathcal{H}_j^D$  is performed by fixed, degree-consistent maps induced by the chosen ordering.

### 13.1 Aliased multiplication operators on $\Pi_n^D$ via Jacobi matrices

Fix a simplex dimension  $D \geq 1$ , Jacobi parameters  $\kappa \in \mathbb{R}^{D+1}$  with  $\kappa_i > -1/2$ , and maximum total degree  $n \geq 0$ . Let

$$\Pi_n^D := \{\text{polynomials in } D \text{ variables of total degree } \leq n\}, \quad M := \dim \Pi_n^D.$$

Let  $\{\phi_{\alpha}^{(\kappa)}\}_{|\alpha| \leq n}$  be the orthonormal Jacobi simplex basis on  $\Pi_n^D$ , ordered in graded lexicographic order (by total degree, then within each homogeneous block). Define the basis column vector

$$\Phi^{(\kappa)}(x) := \begin{bmatrix} \phi_{\alpha_1}^{(\kappa)}(x) \\ \vdots \\ \phi_{\alpha_M}^{(\kappa)}(x) \end{bmatrix} \in \mathbb{R}^M, \quad x \in \Delta^D.$$

Then any  $f \in \Pi_n^D$  has coefficient vector  $c_f \in \mathbb{R}^M$  such that

$$f(x) = (\Phi^{(\kappa)}(x))^T c_f.$$

#### 13.1.1 Jacobi matrices as projected coordinate multipliers

For each coordinate  $x_i$  ( $i = 1, \dots, D$ ) define the multiplication-by-coordinate operator on  $\Pi_n^D$  (with projection back to  $\Pi_n^D$ ) by

$$\mathcal{J}_i^{(n)} := \Pi_n \circ (x_i \cdot) \circ \Pi_n, \quad \mathcal{J}_i^{(n)} : \Pi_n^D \rightarrow \Pi_n^D,$$

where  $\Pi_n$  denotes  $L^2(w_{\kappa})$ -orthogonal projection onto  $\Pi_n^D$ . The corresponding matrix  $J_i \in \mathbb{R}^{M \times M}$  in the basis  $\Phi^{(\kappa)}$  is defined by

$$(J_i)_{\ell k} := \left\langle \phi_{\alpha_{\ell}}^{(\kappa)}, x_i \phi_{\alpha_k}^{(\kappa)} \right\rangle_{w_{\kappa}}, \quad \langle u, v \rangle_{w_{\kappa}} := \int_{\Delta^D} u(x) v(x) w_{\kappa}(x) dx.$$

Equivalently, for any coefficient vector  $c \in \mathbb{R}^M$ ,

$$x_i (\Phi^{(\kappa)}(x))^T c \xrightarrow{\Pi_n} (\Phi^{(\kappa)}(x))^T (J_i c).$$

Since  $x_i$  is real-valued and the basis is orthonormal, each  $J_i$  is symmetric.

### 13.1.2 Aliased multiplication-by- $q$ as a polynomial in $(J_1, \dots, J_D)$

Let  $q$  be a (possibly high-degree) polynomial (or a function approximated by a polynomial) with Jacobi simplex expansion

$$q(x) \approx \sum_{|\beta| \leq p} q_\beta \phi_\beta^{(\kappa)}(x), \quad p \geq 0,$$

and denote by  $c_q \in \mathbb{R}^{\dim \Pi_p^D}$  the coefficient vector  $(q_\beta)$ . We define the *aliased multiplication operator* on  $\Pi_n^D$  associated with  $q$  by

$$\mathcal{M}_q^{(n)} := \Pi_n \circ (q \cdot) \circ \Pi_n : \Pi_n^D \rightarrow \Pi_n^D.$$

In coefficient form,  $\mathcal{M}_q^{(n)}$  is represented by a matrix  $M_q \in \mathbb{R}^{M \times M}$  such that for all  $f \in \Pi_n^D$ ,

$$c_{qf} = M_q c_f, \quad qf \text{ understood as aliased back into } \Pi_n^D.$$

A fundamental representation is obtained by *functional calculus* on the Jacobi matrices:

$$\boxed{M_q = q(J_1, \dots, J_D),}$$

meaning: write  $q$  in any polynomial coordinate representation and substitute  $x_i \mapsto J_i$ . Because  $J_i$  act on  $\Pi_n^D$  and map  $\mathbb{R}^M \rightarrow \mathbb{R}^M$ , the matrix  $q(J_1, \dots, J_D)$  is defined for any polynomial degree, and the resulting operator is automatically aliased to  $\Pi_n^D$ .

**Remark (degree of  $q$ ).** No restriction  $\deg(q) \leq n$  is required to *define*  $q(J_1, \dots, J_D)$ . Increasing the approximation degree  $p$  improves the accuracy of the polynomial surrogate for  $q$ , but does not alter the definition that the output lives in  $\Pi_n^D$ .

### 13.1.3 Why monomial substitution is unstable and why we need operator Clenshaw

If we expand  $q$  in monomials,

$$q(x) = \sum_{|\gamma| \leq p} a_\gamma x^\gamma, \quad x^\gamma := \prod_{i=1}^D x_i^{\gamma_i},$$

then

$$q(J_1, \dots, J_D) = \sum_{|\gamma| \leq p} a_\gamma \prod_{i=1}^D J_i^{\gamma_i},$$

which inherits the ill-conditioning of the monomial basis. This motivates evaluating  $q(J_1, \dots, J_D)$  using a stable recurrence in an orthogonal basis, i.e. an operator-valued Clenshaw-type algorithm.

## 13.2 Sparse *application* $v \mapsto q(J)v$ using only sparse matvecs

In our implementation we do not form  $M_q$  explicitly during assembly-heavy workflows. Instead we implement a routine that applies  $M_q$  to a vector using only:

- sparse matrix–vector products with the CSC Jacobi matrices  $J_i$ , and
- scalar (or small block) recurrences in total degree.

This provides the aliased product  $c_{qf} = q(J)c_f$  at cost comparable to applying a small number of sparse operators.

### 13.2.1 Total-degree block structure

Let  $\mathcal{H}_j^D$  denote the homogeneous subspace of degree exactly  $j$ , with dimension  $m_j := \dim \mathcal{H}_j^D$ . Under graded ordering,

$$\mathbb{R}^M \cong \bigoplus_{j=0}^n \mathbb{R}^{m_j}, \quad c = \begin{bmatrix} c_0 \\ \vdots \\ c_n \end{bmatrix}, \quad c_j \in \mathbb{R}^{m_j}.$$

Each  $J_i$  is block tridiagonal in this decomposition:

$$(J_i)_{j\ell} = 0 \quad \text{unless} \quad \ell \in \{j-1, j, j+1\}.$$

Thus one sparse matvec with  $J_i$  has cost  $\Theta(\text{nnz}(J_i)) = \Theta(M)$ , since  $\text{nnz}(J_i)$  scales linearly with  $M$  for our stencil-built CSC structure.

### 13.2.2 Operator-valued Clenshaw (apply-only form)

Let  $q$  be represented by Jacobi simplex coefficients grouped by total degree,

$$q(x) \approx \sum_{k=0}^p (\Phi_k^{(\kappa)}(x))^T q_k, \quad q_k \in \mathbb{R}^{m_k}.$$

We seek a stable procedure to compute  $y = q(J_1, \dots, J_D)v$  for a given  $v \in \mathbb{R}^M$ .

**Algorithm (apply-only operator Clenshaw).** The algorithm maintains two auxiliary vectors  $b_{k+1}, b_{k+2} \in \mathbb{R}^M$  and performs a backward recurrence in  $k$ :

$$\begin{aligned} b_{p+2} &:= 0, \\ b_{p+1} &:= 0, \\ \text{for } k = p, p-1, \dots, 0 : \quad &b_k := \text{embed}(q_k) + \mathcal{A}_k(J_1, \dots, J_D) b_{k+1} - \mathcal{C}_k b_{k+2}, \end{aligned}$$

and returns

$$y := b_0.$$

Here:

- $\text{embed}(q_k) \in \mathbb{R}^M$  inserts the degree- $k$  coefficient block  $q_k$  into the graded coefficient vector (zeros elsewhere);
- $\mathcal{A}_k(J_1, \dots, J_D)$  denotes a fixed linear combination of the coordinate operators (and possibly degree-dependent shifts) determined by the orthogonal recurrence satisfied by the Jacobi simplex basis; and
- $\mathcal{C}_k$  is a degree-dependent scalar (or block-diagonal-by-degree) coefficient analogous to the univariate  $c_{k+1}$  term in Clenshaw.

Crucially,  $\mathcal{A}_k(J_1, \dots, J_D) b_{k+1}$  is computed using only sparse matvecs:

$$\mathcal{A}_k(J_1, \dots, J_D) b_{k+1} = \sum_{i=1}^D \gamma_{i,k} (J_i b_{k+1}) + \delta_k b_{k+1},$$

for scalar weights  $\gamma_{i,k}, \delta_k$  provided by the chosen Jacobi simplex recurrence. Thus each recurrence step requires  $D$  sparse matvecs and a small number of axpy operations.



**Complexity.** Let  $M = \dim \Pi_n^D = \Theta(n^D)$ . Each sparse matvec  $J_i w$  costs  $\Theta(\text{nnz}(J_i)) = \Theta(M)$ . If the recurrence uses  $D$  matvecs per stage and  $p$  stages (degree- $p$  truncation for  $q$ ), then

$$\text{cost}(v \mapsto q(J)v) = \Theta(p D M) \quad (\text{plus lower-order vector operations}).$$

This matches the intended scaling of sparse operator-valued evaluation schemes: the dominant work is a small number of sparse matvecs per recurrence stage, with no sparse factorization and no fill-in.

**Implementation note.** In the present development phase, we store each  $J_i$  as a CSC matrix and implement  $J_i v$  via a standard CSC SpMV routine. The recurrence coefficients  $(\gamma_{i,k}, \delta_k, \mathcal{C}_k)$  are determined by the simplex Jacobi recurrence used to define the basis. In later sections, we will specialize this construction to the specific  $D$ -simplex Jacobi basis used in our code and provide the explicit coefficient formulas needed to implement the recurrence exactly.

### 13.3 Multiplication-by- $q$ on $\Pi_n^D$ : implementation-level construction

We work throughout in coefficient space on the Jacobi simplex basis  $\{\phi_\alpha^{(\kappa)}\}_{|\alpha| \leq n}$ , ordered by total degree. Let

$$M := \dim \Pi_n^D, \quad c \in \mathbb{R}^M$$

denote the coefficient vector of a function

$$f(x) = \sum_{|\alpha| \leq n} c_\alpha \phi_\alpha^{(\kappa)}(x).$$

#### 13.3.1 Coordinate multiplication operators

For each coordinate  $x_i$  ( $i = 1, \dots, D$ ) we have already constructed the *Jacobi coordinate matrix*

$$J_i \in \mathbb{R}^{M \times M}, \quad (J_i)_{\ell k} = \langle \phi_\ell^{(\kappa)}, x_i \phi_k^{(\kappa)} \rangle_{w_\kappa},$$

stored in compressed sparse column (CSC) format.

These matrices implement *aliased coordinate multiplication*:

$$\Pi_n(x_i f) \longleftrightarrow J_i c.$$

Key structural facts (exploited in code):

- $J_i$  is sparse with  $\text{nnz}(J_i) = \mathcal{O}(M)$ .
- In total-degree block form,

$$\mathbb{R}^M = \bigoplus_{j=0}^n \mathcal{H}_j^D,$$

each  $J_i$  couples only blocks  $j-1$ ,  $j$ , and  $j+1$ .

- Sparse matrix-vector products  $v \mapsto J_i v$  are implemented directly using CSC SpMV.

### 13.3.2 Formal tensor contraction for multiplication by $q$

Let  $q$  be a scalar function with Jacobi simplex expansion

$$q(x) = \sum_{|\beta| \leq p} q_\beta \phi_\beta^{(\kappa)}(x).$$

The *exact* (aliased) multiplication operator  $\mathcal{M}_q^{(n)} = \Pi_n(q) \Pi_n$  has matrix representation

$$(M_q)_{\ell k} = \sum_{|\beta| \leq p} q_\beta \int_{\Delta^D} \phi_\ell^{(\kappa)}(x) \phi_\beta^{(\kappa)}(x) \phi_k^{(\kappa)}(x) w_\kappa(x) dx.$$

Equivalently, if we define the third-order tensor

$$T_{\ell k \beta} = \int_{\Delta^D} \phi_\ell^{(\kappa)}(x) \phi_\beta^{(\kappa)}(x) \phi_k^{(\kappa)}(x) w_\kappa(x) dx,$$

then

$$M_q = \sum_{|\beta| \leq p} q_\beta T_{::\beta}.$$

**Why we do *not* use this directly.** Although  $T_{\ell k \beta}$  is sparse in structured ways, explicitly forming or storing it has the following drawbacks:

- Storage is  $\mathcal{O}(M^2)$  even after pruning.
- Assembling  $M_q$  costs  $\mathcal{O}(pM^2)$  per coefficient update.
- This is prohibitive in model-discovery or repeated-assembly settings.

We therefore do *not* assemble  $M_q$  explicitly.

### 13.3.3 Equivalent operator formulation using Jacobi matrices

The key observation is that  $J_i$  already encode multiplication by  $x_i$  on  $\Pi_n^D$ . Since any polynomial  $q$  can be written as a polynomial in the coordinates,

$$q(x) = Q(x_1, \dots, x_D),$$

we define the aliased multiplication operator by substitution:

$$\boxed{M_q = Q(J_1, \dots, J_D)}.$$

This definition:

- Is algebraically equivalent to the tensor contraction above,
- Automatically includes aliasing to  $\Pi_n^D$ ,
- Requires only the precomputed sparse matrices  $J_i$ .

### 13.3.4 How multiplication by $q$ is *applied* in code

We never form  $M_q$  explicitly. Instead, given a coefficient vector  $c$ , we compute

$$y = M_q c = Q(J_1, \dots, J_D) c$$

using only sparse matrix–vector products.

Algorithmically:

1. Represent  $q$  by its Jacobi coefficients  $\{q_\beta\}_{|\beta| \leq p}$ .
2. Evaluate the matrix polynomial  $Q(J_1, \dots, J_D)$  acting on  $c$  using a stable operator-valued recurrence (Clenshaw-type).
3. Each recurrence step consists only of:
  - sparse SpMV calls  $v \mapsto J_i v$ ,
  - vector additions and scalings.

No dense matrices are formed, and no sparse factorization is performed.

### 13.3.5 Complexity and advantages

Let  $M = \dim \Pi_n^D = \mathcal{O}(n^D)$ . Each sparse multiplication  $v \mapsto J_i v$  costs  $\mathcal{O}(M)$ .

If evaluating  $Q(J_1, \dots, J_D)c$  requires  $\mathcal{O}(pD)$  sparse matvecs (as in operator-valued Clenshaw), then

$$\text{cost}(c \mapsto q(J)c) = \mathcal{O}(pD M).$$

This has several decisive advantages:

- Linear scaling in the number of coefficients.
- No fill-in (no sparse elimination).
- Ideal for repeated operator application in PDE solves and model discovery.
- Fully compatible with CSC storage and existing SpMV kernels.

**Summary.** Although multiplication by  $q$  can be written as a three-index tensor contraction, we implement it in practice as a sequence of sparse matrix–vector products with the Jacobi coordinate matrices  $J_i$ . This matches both the algebraic structure of Jacobi polynomials on the simplex and the performance requirements of assembly-heavy and data-driven PDE solvers.

## 13.4 Multiplication-by- $q$ on $\Pi_n^D$ and an operator-valued Clenshaw recurrence

**Spaces and ordering.** Fix  $D \geq 1$ , parameters  $\kappa \in \mathbb{R}^{D+1}$  with  $\kappa_i > -1/2$ , and degree cutoff  $n \geq 0$ . Let  $\Pi_n^D$  be the space of polynomials of total degree  $\leq n$  on the simplex  $\Delta^D$  with the orthonormal Jacobi simplex basis  $\{\phi_\alpha^{(\kappa)}\}_{|\alpha| \leq n}$  ordered by total degree. Write

$$M := \dim \Pi_n^D, \quad \mathbb{R}^M \cong \bigoplus_{j=0}^n \mathbb{R}^{m_j}, \quad m_j := \dim \mathcal{H}_j^D,$$

where  $\mathcal{H}_j^D$  denotes the homogeneous subspace of degree exactly  $j$ . We write any coefficient vector  $c \in \mathbb{R}^M$  in blocks:

$$c = \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix}, \quad c_j \in \mathbb{R}^{m_j}.$$

#### 13.4.1 Coordinate Jacobi matrices and their degree-block extraction

For each coordinate  $x_i$  ( $i = 1, \dots, D$ ) we have the Jacobi (coordinate-multiplication) matrix

$$J_i \in \mathbb{R}^{M \times M}, \quad (J_i)_{\ell k} = \left\langle \phi_\ell^{(\kappa)}, x_i \phi_k^{(\kappa)} \right\rangle_{w_\kappa},$$

stored in CSC format. These implement aliased multiplication by  $x_i$  on  $\Pi_n^D$ :

$$\Pi_n(x_i f) \longleftrightarrow J_i c_f.$$

Let  $E_j \in \mathbb{R}^{M \times m_j}$  denote the block embedding that inserts a degree- $j$  block into the global vector (zeros elsewhere), and  $E_j^T$  the corresponding extraction:

$$c_j = E_j^T c, \quad c = \sum_{j=0}^n E_j c_j.$$

From the full CSC matrix  $J_i$  we extract the three degree-coupling blocks:

$$\begin{aligned} U_{i,j} &:= E_{j+1}^T J_i E_j \in \mathbb{R}^{m_{j+1} \times m_j} && \text{(up block),} \\ B_{i,j} &:= E_j^T J_i E_j \in \mathbb{R}^{m_j \times m_j} && \text{(same-degree block),} \\ L_{i,j} &:= E_{j-1}^T J_i E_j \in \mathbb{R}^{m_{j-1} \times m_j} && \text{(down block).} \end{aligned}$$

In code,  $E_j^T J_i E_j$  means: restrict to rows/cols belonging to the degree- $j$  index range.

**Degree-tridiagonal structure.** Because multiplication by a coordinate changes total degree by at most 1,

$$(J_i)_{j\ell} = 0 \quad \text{unless} \quad \ell \in \{j-1, j, j+1\},$$

so each  $J_i$  is block tridiagonal in total degree, and  $U_{i,j}, B_{i,j}, L_{i,j}$  are the only nonzero block bands.

#### 13.4.2 A vector three-term recurrence in total degree (constructed from the $J_i$ )

Let  $\Phi_j(x) \in \mathbb{R}^{m_j}$  denote the vector of degree- $j$  basis functions (the homogeneous block of  $\Phi^{(\kappa)}$ ). Then the definition of  $J_i$  implies the operator identity on  $\Pi_n^D$ :

$$x_i \Phi_j(x) = \Phi_{j+1}(x) U_{i,j}^T + \Phi_j(x) B_{i,j}^T + \Phi_{j-1}(x) L_{i,j}^T, \quad j \geq 0,$$

with the convention  $\Phi_{-1} \equiv 0$ .

This is the multivariate analogue of the univariate three-term recurrence, and importantly: *we do not derive closed-form coefficients*; we *extract* these blocks directly from the precomputed Jacobi matrices  $J_i$ .

### 13.4.3 Multiplication-by- $q$ as an operator polynomial in $\{J_i\}$

Let  $q$  be represented in the same simplex basis (possibly at degree  $p \geq n$ ):

$$q(x) \approx \sum_{j=0}^p \Phi_j(x)^T q_j, \quad q_j \in \mathbb{R}^{m_j}.$$

We define the aliased multiplication operator on  $\Pi_n^D$  by

$$\mathcal{M}_q^{(n)} := \Pi_n \circ (q \cdot) \circ \Pi_n,$$

whose coefficient action is

$$y = M_q c, \quad y, c \in \mathbb{R}^M.$$

Because  $J_i$  represent coordinate multiplication on  $\Pi_n^D$ , we may equivalently define

$$\boxed{M_q := q(J_1, \dots, J_D),}$$

which is an  $M \times M$  linear operator acting on coefficient vectors. This definition automatically aliases to  $\Pi_n^D$  since each  $J_i$  maps  $\mathbb{R}^M \rightarrow \mathbb{R}^M$ .

**Why we do not assemble  $M_q$ .** The triple-product tensor contraction

$$(M_q)_{\ell k} = \sum_{\beta} q_{\beta} \int \phi_{\ell} \phi_{\beta} \phi_k w_{\kappa}$$

is correct but assembling/storing  $M_q$  for each new  $q$  costs  $\mathcal{O}(M^2)$  and is unsuitable for repeated-assembly workflows (e.g. model discovery). Instead we implement the *apply-only* map  $c \mapsto M_q c$ .

## 13.5 Operator-valued Clenshaw: apply $y = q(J)c$ using only sparse SpMV with $J_i$

### 13.5.1 Stacked raising operator and its left-inverse (OTV-style)

Define the *stacked raising operator* at degree  $j$  by

$$\mathcal{U}_j := \begin{bmatrix} U_{1,j} \\ \vdots \\ U_{D,j} \end{bmatrix} \in \mathbb{R}^{(D m_{j+1}) \times m_j}.$$

This is the  $D$ -dimensional analogue of the stacked  $[B_n^x; B_n^y]$  operator in OTV. We assume  $\mathcal{U}_j$  has full column rank (true in the Jacobi simplex setting away from low-degree degeneracies), so we can define a left-inverse (minimum-norm) map

$$\mathcal{U}_j^+ \in \mathbb{R}^{m_j \times (D m_{j+1})}, \quad \mathcal{U}_j^+ \mathcal{U}_j = I_{m_j}.$$

*Implementation:* we precompute a SuiteSparseQR factorization of  $\mathcal{U}_j$  once per  $(D, n, \kappa)$ , and apply  $\mathcal{U}_j^+$  via the cached factors.

### 13.5.2 A degree-recursion that replaces explicit recurrence derivation

From the block identity

$$x_i \Phi_j = \Phi_{j+1} U_{i,j}^T + \Phi_j B_{i,j}^T + \Phi_{j-1} L_{i,j}^T,$$

stack over  $i = 1, \dots, D$  and rearrange to isolate  $\Phi_{j+1}$ :

$$\begin{bmatrix} x_1 \Phi_j \\ \vdots \\ x_D \Phi_j \end{bmatrix} - \begin{bmatrix} \Phi_j B_{1,j}^T + \Phi_{j-1} L_{1,j}^T \\ \vdots \\ \Phi_j B_{D,j}^T + \Phi_{j-1} L_{D,j}^T \end{bmatrix} = \begin{bmatrix} \Phi_{j+1} U_{1,j}^T \\ \vdots \\ \Phi_{j+1} U_{D,j}^T \end{bmatrix}.$$

Equivalently,

$$\begin{bmatrix} x_1 \Phi_j \\ \vdots \\ x_D \Phi_j \end{bmatrix} - \mathcal{B}_j(\Phi_j, \Phi_{j-1}) = (I_D \otimes \Phi_{j+1}) \mathcal{U}_j^T,$$

where  $\mathcal{B}_j$  denotes the known (extracted) lower-degree contributions. Right-multiplying by  $\mathcal{U}_j^{+T}$  yields an explicit formula for  $\Phi_{j+1}$ :

$$\Phi_{j+1} = \left( \begin{bmatrix} x_1 \Phi_j \\ \vdots \\ x_D \Phi_j \end{bmatrix} - \mathcal{B}_j(\Phi_j, \Phi_{j-1}) \right)^T \mathcal{U}_j^{+T}.$$

*Crucial point:* this is a recurrence in total degree that uses only the blocks extracted from  $J_i$  and the left-inverse  $\mathcal{U}_j^+$ ; we never need symbolic recurrence coefficients.

### 13.5.3 Operator Clenshaw for applying $q(J)$ to a vector

We now derive the apply-only recurrence for

$$y = q(J_1, \dots, J_D)c, \quad c \in \mathbb{R}^M.$$

Write  $q$  in homogeneous blocks  $q_j \in \mathbb{R}^{m_j}$ :

$$q(x) \approx \sum_{j=0}^p \Phi_j(x)^T q_j.$$

Define the embedding of a degree- $j$  coefficient block into  $\mathbb{R}^M$  by

$$\text{emb}_j(q_j) := E_j q_j \in \mathbb{R}^M.$$

**Backward recurrence states.** We compute auxiliary vectors  $b_j \in \mathbb{R}^M$  for  $j = p+1, p, \dots, 0$  by a backward sweep:

$$b_{p+2} = 0, \quad b_{p+1} = 0.$$

For  $j = p, p-1, \dots, 0$  we update

$$b_j = \text{emb}_j(q_j) \odot c + \sum_{i=1}^D J_i \Gamma_{i,j}(b_{j+1}) - \Delta_{j+1}(b_{j+2}), \quad (22)$$

and return

$$y := b_0.$$

**What are  $\Gamma_{i,j}$  and  $\Delta_j$ ? (No symbolic recurrences.)** In univariate Clenshaw,  $\Gamma$  and  $\Delta$  are scalar multiples determined by the 3-term recurrence. Here, instead of closed forms, we determine the needed “degree-propagation” maps directly from the degree blocks extracted from  $J_i$  and the left-inverses  $\mathcal{U}_j^+$ .

Concretely, define degree-extraction of any global vector  $v \in \mathbb{R}^M$  as  $v_j = E_j^T v$ . Then we define the degree- $j$  components of (22) by enforcing that, for each  $j$ , the contribution from  $b_{j+1}$  is mapped through the same up/same/down splitting as coordinate multiplication. This is achieved by:

- applying SpMV with each  $J_i$  to  $b_{j+1}$  (global vectors), and
- using the cached  $\mathcal{U}_j^+$  to convert the stacked degree- $(j+1)$  contributions into the correct degree- $j$  block update.

**Implementation form (blockwise).** Let  $z_i := J_i b_{j+1}$  for  $i = 1, \dots, D$  (computed by CSC SpMV). Extract their degree- $(j+1)$  blocks and stack:

$$Z_{j+1} := \begin{bmatrix} E_{j+1}^T z_1 \\ \vdots \\ E_{j+1}^T z_D \end{bmatrix} \in \mathbb{R}^{D m_{j+1}}.$$

Then the recurrence uses the precomputed left-inverse to fold this stacked object into a degree- $j$  update:

$$\tilde{b}_j := E_j \left( \mathcal{U}_j^+ Z_{j+1} \right) \in \mathbb{R}^M.$$

Finally incorporate the lower-degree correction (the analogue of the  $c_{j+1} b_{j+2}$  term in 1D), again using blocks extracted from  $J_i$ :

$$\Delta_{j+1}(b_{j+2}) := \sum_{i=1}^D E_j \left( L_{i,j+1} (E_{j+2}^T b_{j+2}) \right),$$

with admissibility conventions at low degrees.

Putting this together, one practical (apply-only) realization of (22) is:

$$b_j = E_j q_j \odot c + E_j \left( \mathcal{U}_j^+ \begin{bmatrix} E_{j+1}^T (J_1 b_{j+1}) \\ \vdots \\ E_{j+1}^T (J_D b_{j+1}) \end{bmatrix} \right) - \sum_{i=1}^D E_j \left( L_{i,j+1} (E_{j+2}^T b_{j+2}) \right), \quad (23)$$

and we return  $y = b_0$ .

**Notes.**

- The only expensive primitives are SpMV calls with  $J_i$  and applications of  $\mathcal{U}_j^+$ .
- All maps  $E_j$ ,  $U_{i,j}$ ,  $L_{i,j}$  are obtained from the degree-block structure of the stored CSC  $J_i$ .
- The left-inverse applications  $\mathcal{U}_j^+$  are implemented via cached SuiteSparseQR factors.

#### 13.5.4 Complexity (matches OTV-style scaling)

Let  $M = \dim \Pi_n^D = \Theta(n^D)$  and assume  $\text{nnz}(J_i) = \Theta(M)$  (true for our stencil-built  $J_i$ ). Each SpMV  $J_i v$  costs  $\Theta(M)$ .

A degree-sweep over  $j = 0, \dots, p$  performs:

- $D$  SpMV operations per level (to form  $J_i b_{j+1}$ ),
- one application of  $\mathcal{U}_j^+$  per level (small: acts on dimension  $Dm_{j+1}$ ),
- lower-order block extractions and axpy operations.

Thus the total cost to apply  $y = q(J)c$  is

$$\text{cost}(c \mapsto q(J)c) = \Theta(p D M) + \sum_{j=0}^p \text{cost}(\mathcal{U}_j^+ \text{-apply}),$$

with *no fill-in* in the operators (no elimination is performed during application). All fill-in occurs only in the one-time QR factorization of  $\mathcal{U}_j$  (offline), which is cached.

**Why this is preferable to precomputing  $M_q$ .** Forming  $M_q$  explicitly requires at least  $\Theta(M^2)$  storage and  $\Theta(M^2)$  work per update. In contrast, the apply-only Clenshaw recurrence (23) uses only  $\Theta(pDM)$  work, making it suitable for repeated solves and model discovery where  $q$  changes frequently.